

Introdução

Ao se trabalhar com programação, principalmente no contexto da robótica, é comum precisar que uma operação seja realizada várias vezes.

Seja na checagem do estado de um sensor, na construção de um mapa ou na utilização ou na utilização de algoritmos de otimização, os chamados **Loops** estão presentes nas mais diversas aplicações.

Abaixo vamos aprender sobre os laços de repetição **While**, **do While** e **For**, que constituem o "arsenal" de loops no C++.

```
In [1]: # Rode o comando abaixo para poder usar os códigos em cpp

!mkdir cpp_files
```

mkdir: cannot create directory 'cpp_files': File exists

While

O bloco de **while** realizará o código contido dentro dele de maneira cíclica enquanto a condição estabelecida em sua declaração for *verdadeira*. Sua sintaxe é a mostrada abaixo.

```
while(CONDIÇÃO){
    (CÓDIGO A SER EXECUTADO)
}
```

Abaixo podemos ver um exemplo simples de aplicação. Sugiro que altere o valor de **i** e estude como isso altera o código. (Dica, teste um $i \geq 10$)

```
In [2]: %%writefile cpp_files/primeiro_exemplo.cpp
#include <iostream>
using namespace std;

int main(){
    int i = 5;
    cout<<"Estou no inicio, meu i inicial eh: "<<i<<endl;
    while(i<10){
        cout<<"Estou na iteração "<<i<<endl;
        i++;
    }

    cout<<"Estou fora do while"<<endl;
    return 0;
}
```

Overwriting cpp_files/primeiro_exemplo.cpp

In [3]: `!g++ cpp_files/primeiro_exemplo.cpp -o cpp_files/primeiro_exemplo`

do While

Você deve ter notado no exemplo anterior que caso `i` fosse maior do que 10 o laço de repetição não foi executado. Isso ocorre porque o "passo a passo" do while é **Verificação da condição** -> **Execução**, e, caso a condição seja falsa, a execução não acontecerá.

O **do While** se diferencia por ser uma variante do **While** na qual o bloco de código será executado **uma vez** antes da verificação, seguindo posteriormente com o ciclo, caso a condição seja verdadeira, ou parando, caso a condição seja falsa.

A estrutura de um **do While** é a seguinte:

```
do{
    (CODIGO A SER EXECUTADO)
}while(CONDIÇÃO);
```

Nota: o ponto e vírgula no final da estrutura do **do While** é **OBRIGATÓRIO**, com sua ausência resultando em erro.

A vantagem desse método é a realização de uma ação que você sabe que precisa acontecer pelo menos uma vez, como na verificação de um estado. Para isso, você precisaria necessariamente verificar uma vez, o que não seria possível com a estrutura regular do while. Abaixo está o código fictício para o exemplo acima.

```
String estado = "not ready"
```

```
do{
    estado = checaEstado();
}while(estado!="ready");
```

Note que, caso tivéssemos um while regular, a operação `checaEstado()` nunca seria executada, portanto a variável `estado` nunca mudaria seu valor de `not ready` para `ready`. Rode o exemplo abaixo e veja o comportamento do código nas condições de `i` menor, `i` igual ou `i` maior do que 5.

```
In [4]: %%writefile cpp_files/segundo_exemplo.cpp
#include <iostream>
using namespace std;

int main(){

    int i = 0;
    do{
        cout << "i is " << i << "\n";
        i++;
    }while (i < 5);
```

```
return 0;
}
```

Overwriting cpp_files/segundo_exemplo.cpp

```
In [5]: !g++ cpp_files/segundo_exemplo.cpp -o cpp_files/segundo_exemplo
```

For

O laço for é uma estrutura de repetição usada quando o número de iterações é conhecido, ou quando existe uma variável de controle que evolui de forma previsível a cada ciclo. Em outras palavras, ele é especialmente adequado para situações em que queremos repetir uma ação acompanhando explicitamente um contador, um índice ou uma sequência de valores. O esqueleto de um for é o seguinte:

```
for(inicializacao; condicao; atualizacao){
    // bloco de código
}
```

Onde:

- **Inicialização** : executada uma única vez, no início do laço. Normalmente define a variável de controle.
- **Condição** : testada antes de cada iteração. Enquanto for verdadeira, o laço continua.
- **Atualização** : executada ao final de cada repetição, geralmente alterando a variável de controle.

Vamos analisar um exemplo real utilizando **for** :

```
In [ ]: %%writefile cpp_files/terceiro_exemplo.cpp
#include <iostream>
using namespace std;

int main(){

    for(int i = 0; i < 5; i++){
        cout<<"Iteração: "<<i<<endl;
    }

    return 0;
}
```

Writing cpp_files/terceiro_exemplo.cpp

```
In [36]: !g++ cpp_files/terceiro_exemplo.cpp -o cpp_files/terceiro_exemplo
```

O fluxo do código abaixo é o seguinte:

1. `int i = 0` é executado uma única vez, iniciando a variável `i` com 0, servindo como contador.
2. O programa verifica se `i < 5`.

3. Como a condição é verdadeira, o bloco é executado, imprimindo na tela o número da iteração.
4. Ao final do bloco, `i++` é executado.
5. A condição é verificada novamente.
6. O processo se repete até que a condição se torne falsa.

Vamos analisar um exemplo mais usual no contexto da robótica, com um sensor fictício de distância realizando medições ao longo de uma faixa de ângulos.

```
In [13]: %%writefile cpp_files/quarto_exemplo.cpp
#include <iostream>
#include <cstdlib>
using namespace std;

int main() {

    int max = 20, min = 0;

    for(int angulo = 0; angulo <= 180; angulo += 30){
        cout << "Posicionando sensor em " << angulo << " graus" << endl;
        int dist = rand() % (max-min) + min;
        cout << "Distância medida: " << dist << endl<<endl;

    }

    return 0;
}
```

Overwriting cpp_files/quarto_exemplo.cpp

```
In [37]: !g++ cpp_files/quarto_exemplo.cpp -o cpp_files/quarto_exemplo
```

Nested loops

Chamamos de `Nested Loops`, ou loops aninhados, blocos de códigos que contêm um loop dentro do código a ser executado de outro. São muito utilizados quando se trabalha com matrizes (que estudaremos na aula seguinte). Para utilizar loops aninhados, é importante entender a ordem de execução dos blocos de código, que ocorre "de dentro para fora".

Vamos analisar o exemplo abaixo:

```
In [28]: %%writefile cpp_files/quinto_exemplo.cpp
#include <iostream>
using namespace std;

int main() {

    for(int i = 0; i < 3; i++){
        for(int j = 0; j < 3; j++){
```

```

        cout<<"Valores da execução: j = "<<j<<" | i = "<<i<<endl;
    }
    cout << endl;
}

return 0;
}

```

Overwriting cpp_files/quinto_exemplo.cpp

In [38]: `!g++ cpp_files/quinto_exemplo.cpp -o cpp_files/quinto_exemplo`

Podemos notar que a ordem de execução está seguindo: Em cada iteração do loop externo (com iterador i), realiza-se todo o loop interno (com iterador j).

Vamos ver um exemplo de aplicação, onde

In [43]:

```

%%writefile cpp_files/sexta_exemplo.cpp
#include <iostream>
using namespace std;

int main() {

    int num_ciclos = 0;
    cout<<"Insira o numero de ciclos: ";
    cin>>num_ciclos;

    for(int ciclo = 1; ciclo <= num_ciclos; ciclo++){

        cout << "Ciclo " << ciclo << " iniciado" << endl;

        bool executando = true;
        string comando;
        int num_comandos = 0;

        while(executando){

            cout << "Digite um comando (parar para encerrar): ";
            cin >> comando;

            if(comando == "parar"){
                cout << "Encerrando ciclo..." << endl;
                executando = false;
            } else {
                cout << "Comando recebido: " << comando << endl;
            }
            num_comandos++;
        }
        cout<<endl<< "No ciclo "<<ciclo<<" foram executados "<<num_comandos<

    }

    return 0;
}

```

Overwriting cpp_files/sexta_exemplo.cpp

```
In [44]: !g++ cpp_files/sextto_exemplo.cpp -o cpp_files/sextto_exemplo
```

Break e Continue

Durante a execução de loops, nem sempre queremos seguir o fluxo padrão até o final. Em muitos casos, precisamos interromper o loop antes do previsto ou pular apenas uma parte da execução.

Para isso, utilizamos duas instruções importantes:

- `break`
- `continue`

Essas instruções alteram o fluxo do programa dentro de estruturas de repetição.

Break

O comando `break` é utilizado para encerrar imediatamente um loop. Assim que ele é executado, o programa sai do laço e continua a execução a partir da próxima instrução após o loop. Isso significa que o restante das iterações não será executado.

O `break` é útil quando:

- Você encontrou o resultado que estava procurando;
- Uma condição de erro ocorreu;
- Não faz mais sentido continuar a repetição.

Vamos analisar o exemplo abaixo:

```
In [46]: %writefile cpp_files/setimo_exemplo.cpp
#include <iostream>
using namespace std;

int main() {
    int numero;

    while(true){
        cout << "Digite um numero (0 para sair): ";
        cin >> numero;

        if(numero == 0){
            break;
        }

        cout << "Voce digitou: " << numero << endl;
    }
}
```

```
    cout << "Programa encerrado." << endl;

    return 0;
}
```

Writing cpp_files/setimo_exemplo.cpp

```
In [47]: !g++ cpp_files/setimo_exemplo.cpp -o cpp_files/setimo_exemplo
```

Podemos notar que, ao digitarmos 0, o código sai do loop, conforme o esperado.

Continue

O comando continue é utilizado para interromper a iteração atual de um loop e passar imediatamente para a próxima. Assim que ele é executado, o restante do código dentro daquela iteração não será executado, mas o loop continua normalmente.

O continue é útil quando:

- você quer ignorar casos específicos dentro do loop;
- existem dados inválidos que não devem ser processados;
- apenas parte das iterações deve executar todo o código.

```
In [53]: %%writefile cpp_files/oitavo_exemplo.cpp
#include <iostream>
using namespace std;

int main() {

    for(int leitura = 1; leitura <= 6; leitura++){

        int valor;
        cout << "Digite a leitura do sensor " << leitura << ": ";
        cin >> valor;

        if(valor < 0){
            cout << "Leitura invalida, ignorando..." << endl;
            continue;
        }

        cout << "Processando valor: " << valor << endl;
    }

    return 0;
}
```

Overwriting cpp_files/oitavo_exemplo.cpp

```
In [54]: !g++ cpp_files/oitavo_exemplo.cpp -o cpp_files/oitavo_exemplo
```

Resumo e finalização

Nesta aula aprendemos como controlar a repetição de código usando estruturas de loop.

Vimos como o C++ permite executar blocos de código múltiplas vezes com:

- while – repetir enquanto uma condição for verdadeira
- do-while – garantir que o código execute pelo menos uma vez
- for – repetir com controle explícito de inicialização, condição e atualização
- nested loops – utilizar loops dentro de outros para trabalhar com estruturas mais complexas
- break – encerrar um loop imediatamente
- continue – pular uma iteração específica e seguir para a próxima

Essas estruturas são fundamentais para sistemas que precisam operar continuamente, como leitura de sensores, controle de movimento e processamento de dados em robótica com C++ + ROS.

Faça a lista de exercícios desta aula e não hesite em tirar dúvidas!